

# DATA SENTINEL

Self-Observing. Self-Verifying. Graph-Powered.

**Architecture Contract:** CSV → FastAPI → Apache Kafka → Loader → Neo4j → Grounded Chat

**Core Story:** "Data Sentinel transforms unknown CSV data into a live, queryable graph while continuously observing the ingestion pipeline and verifying every answer against actual Neo4j evidence."

---

## 1. Executive Summary

Modern enterprise organizations receive recurring, ad-hoc spreadsheet and CSV data from diverse operational systems including CRMs, billing systems, logistics partners, and ticketing platforms. Traditionally, extracting business intelligence from these flat files requires manual spreadsheet inspection, bespoke database migrations, or opaque AI wrappers that frequently hallucinate facts.

**Data Sentinel** resolves this operational bottleneck by introducing an automated, fully decoupled ingestion and intelligence pipeline:

**MANDATORY PIPELINE:** CSV → FastAPI (8000) → Kafka Topic (csv-rows) → Loader Consumer → Neo4j (7474) → Deterministic Chat

Data Sentinel does not require predefined schemas or database administration. Beyond foundational ingestion, it introduces three tightly scoped engineering innovations: (1) **Live Telemetry Engine** synchronizing backend streaming state directly into 3D React Three Fiber visualizations; (2) **Sentinel Verification & Anti-Hallucination Firewall** providing an auditable 6-stage verification trail and explicitly blocking ungrounded queries; and (3) **Visible Cross-Dataset Relationship Intelligence** that discovers shared entity linkages with factual match bases.

## 2. Problem Statement

The hackathon challenge mandates a reliable, decoupled system where users upload arbitrary CSV files and query them in plain English without manual intermediate processing. The mandatory constraints address real-world ingestion challenges:

- **Decoupled Streaming (Kafka):** Direct API-to-database insertion violates reliability; backpressure buffers are required to absorb ingestion spikes and prevent database connection exhaustion.
- **Graph Storage (Neo4j):** Tabular data contains implicit entity graphs (repeating customer IDs, shared accounts). Relational tables hide these linkages behind expensive joins, whereas graph databases represent relationships natively.
- **Verifiable Grounding:** Enterprise decision-makers cannot trust black-box LLMs that fabricate answers. Every query answer must be strictly traceable to Neo4j graph evidence.

### 3. Existing Approaches and Industry Capabilities

To properly establish the uniqueness of Data Sentinel, we distinguish between capabilities already standard in industry and our novel integrated architecture:

| Existing Technology  | Primary Capability Provided                | Limitation in This Problem Domain                                     |
|----------------------|--------------------------------------------|-----------------------------------------------------------------------|
| CSV / Spreadsheets   | Universal data exchange across platforms   | Manual parsing required; lacks relational integrity and querying      |
| Relational DB (SQL)  | Structured storage, ACID transactions      | Rigid DDL schemas; expensive multi-hop joins on dynamic data          |
| Apache Kafka         | Distributed pub/sub streaming & buffering  | Transport layer only; provides no graph querying or NL answers        |
| Neo4j Graph Database | Native node & relationship storage, Cypher | Requires external ingestion pipeline and schema translation           |
| LLM Data Assistants  | Flexible natural language conversation     | Prone to hallucinations, statistical guessing, and security leakage   |
| Rule-Based Chatbots  | Deterministic query translation            | Typically fragile and opaque; lacks verifiable evidence audit trails  |
| BI Dashboards        | Pre-aggregated charts and metrics          | Requires upfront data modeling; inflexible for unknown CSV structures |

### 4. Gap Identification & Our Proposed Solution

While Kafka, Neo4j, and FastAPI exist individually, unifying them into an automated, self-observing pipeline reveals major engineering gaps in practice: (1) Ingestion pipelines are black boxes that hide buffering and failure states; (2) Re-uploading data creates corrupted duplicates; and (3) LLM interfaces cannot guarantee truthfulness. **Data Sentinel** resolves these gaps through five integrated architectural tiers:

| Tier     | Component               | Implementation Details                       | Engineering Role                                    |
|----------|-------------------------|----------------------------------------------|-----------------------------------------------------|
| Client   | React / Vite / Three.js | TypeScript, React Three Fiber, TailwindCSS   | 3D pipeline telemetry, chat, graph visualization    |
| Gateway  | FastAPI (Python 3.11)   | Pydantic v2, SHA-256 identity, rate limiting | CSV validation, Kafka producer, status calculations |
| Conveyor | Apache Kafka 3.7.0      | KRaft mode, topic: csv-rows                  | Decoupled streaming buffer; absorbs backpressure    |
| Loader   | Python Consumer         | Confluent-kafka consumer, Neo4j driver       | Consumes rows, executes atomic MERGE queries        |
| Storage  | Neo4j 5.20 Community    | Native property graph, constraints, indexes  | System of record for datasets, rows, and linkages   |

## 5. End-to-End Data Flow

The operational lifecycle of an ingestion event flows through 16 strictly verified stages:

1. **Ingestion Initiation:** User selects an unfamiliar CSV file and submits it via the React client.
2. **Payload Validation:** FastAPI POST /ingest validates MIME type, non-empty bytes, and header existence.
3. **Deterministic Identity:** System computes `SHA256(raw_bytes)[:16]` as the invariant `dataset_id`.
4. **Metadata Registration:** `(:Dataset)` node created in Neo4j with status 'loading' and timestamp.
5. **Kafka Streaming:** CSV rows are published as JSON payloads to Kafka topic `csv-rows`.
6. **Producer Flush:** Producer blocks until all broker receipts are acknowledged before returning HTTP 200.
7. **Consumer Dequeue:** Independent Loader consumer fetches batches of row messages from Kafka.
8. **Idempotent Graph Write:** Loader executes Cypher MERGE on composite key `(dataset_id, row_index)`.
9. **Failure Accounting:** Serialization errors increment persisted `d.rows_failed` on Dataset node.
10. **Live Polling:** Frontend queries GET /status?job\_id=... at 1000ms intervals.
11. **Metric Derivation:** API calculates throughput (rows/sec) and duration from real database timestamps.
12. **Ingestion Completion:** When `loaded + failed == total`, status transitions to 'complete' and polling stops.
13. **Question Input:** User asks a plain English question via the Chat interface.
14. **Intent Classification:** Deterministic classifier extracts entities, columns, and target filters.
15. **Graph Execution:** Cypher executes against Neo4j; evidence result set is validated.
16. **Sentinel Verification:** 6-stage audit trail renders in UI; unsupported queries are blocked.

## 6. Graph Data Model & Idempotency Guarantee

Data Sentinel utilizes a dynamic, schema-agnostic graph model that accommodates any tabular data:

```
(:Dataset) {id: "c1a4ac1a4bcbeb82", filename: "demo.csv", row_count: 100, rows_failed: 0, rows_read: 100}
      |
      HAS_ROW
      v
(:Row) {dataset_id: "c1a4ac1a4bcbeb82", row_index: 0, customer_id: "C001", department: "Billing", city:
"Chennai", amount: 5000}
```

**Mathematical Idempotency Guarantee:** The system enforces unique node keys on `(r.dataset_id, r.row_index)`. Because `dataset_id` is a deterministic SHA-256 hash of the exact file bytes, uploading the exact same CSV twice produces identical hashes and row indexes. The Loader executes `MERGE (r:Row {dataset_id: $id, row_index: $idx}) SET r += $props`, matching existing nodes without creating duplicates. Total row count remains strictly invariant.

## 7. Chatbot Engine & Sentinel Verification Firewall

Data Sentinel intentionally rejects external LLMs in favor of a **deterministic natural-language-to-Cypher translation engine**. In enterprise data operations, an ungrounded hallucination is catastrophic; deterministic Cypher guarantees that every answer is backed by database proof.

| Stage | Pipeline Stage Name | Verification Action Executed                             | Enterprise Assurance         |
|-------|---------------------|----------------------------------------------------------|------------------------------|
| 01    | QUESTION RECEIVED   | Logs exact incoming natural language text                | Immutable query audit log    |
| 02    | INTENT DETECTED     | Deterministic AST classification (category, count, etc.) | Zero statistical ambiguity   |
| 03    | CYPHER GENERATED    | Generates parameterized, executable Cypher query         | Complete query transparency  |
| 04    | NEO4J EXECUTED      | Executes query against active Neo4j graph cluster        | Real-time database execution |
| 05    | EVIDENCE FOUND      | Validates non-empty result record set from database      | Empirical evidence required  |
| 06    | ANSWER VERIFIED     | Marks grounded: true and generates human answer          | Certified verifiable answer  |

### Anti-Hallucination Firewall Rule

If a question cannot be answered from the uploaded dataset (e.g., *"Who was the Prime Minister of the UK in 1985?"*), the chatbot **must never fabricate an answer**. The Firewall intercepts the query, returning grounded: false, cypher: "", result: [], and the factual message: *"I don't have that information in the uploaded data."* The UI renders an explicit red **[ANSWER BLOCKED]** banner.

| VERIFIED QUERY DEMO                                                                                                                                                                                                                                                                                                                                  | UNSUPPORTED QUERY FIREWALL BLOCK                                                                                                                                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Question:</b> "How many rows belong to Billing?"</p> <p><b>Intent:</b> Filtered category count</p> <p><b>Cypher:</b> MATCH ... WHERE r.`category` = 'Billing'<br/>RETURN count(r)</p> <p><b>Neo4j Result:</b> [{"count": 47}]</p> <p><b>Status:</b> ✓ GROUNDED IN DATA</p> <p><b>Answer:</b> "There are 47 rows where category = Billing."</p> | <p><b>Question:</b> "Who was the Prime Minister in 1985?"</p> <p><b>Intent:</b> Outside dataset schema</p> <p><b>Cypher:</b> "" (Blocked)</p> <p><b>Neo4j Result:</b> []</p> <p><b>Status:</b> ✗ OUTSIDE DATASET · ANSWER BLOCKED</p> <p><b>Answer:</b> "I don't have that information in the uploaded data."</p> |

## 8. Innovation #1 — Live Telemetry Engine

Traditional data ingestion systems hide pipeline state behind static loading spinners. Data Sentinel exposes a **Live Telemetry Engine** where all displayed numbers originate directly from verified backend state:

| Pipeline Metric | Backend Source of Truth                     | Operational Significance                                   |
|-----------------|---------------------------------------------|------------------------------------------------------------|
| ROWS READ       | Stream parsed lines in FastAPI /ingest      | Confirms total volume extracted from source CSV            |
| KAFKA MESSAGES  | Receipt acknowledgments from producer flush | Confirms row messages committed to Kafka broker            |
| ROWS PROCESSED  | Neo4j count(r:Row) matching dataset_id      | Verifies consumption by Loader and graph ingestion         |
| NEO4J COMMITTED | Active Row nodes written via MERGE          | Validates graph persistence and queryability               |
| FAILED / DLQ    | Persisted d.rows_failed property on Dataset | Surfaces malformed records branching to Dead Letter Queue  |
| THROUGHPUT      | rows_loaded / duration_sec                  | Measures real-time ingestion velocity (e.g. 48.5 rows/sec) |
| DURATION        | (completed_at - started_at) * 1000          | High-resolution execution duration (e.g. 2000.1 ms)        |

## 9. Innovation #3 — Cross-Dataset Relationship Intelligence

When multiple CSV files are uploaded (e.g., customers.csv and orders.csv), Data Sentinel automatically discovers shared entity relationships across distinct datasets without requiring manual foreign key definitions.

**Factual Match Basis:** The discovery engine identifies overlapping identifier keys (customer\_id, order\_id, account\_id) and calculates exact intersection counts. The system reports factual proofs (e.g., *Matched on 3 shared customer\_id values between Customers.csv and Orders.csv*) without claiming artificial statistical confidence percentages.

## 10. Data Health Engine & Anomaly Detection

Data Sentinel computes a composite **Data Health Score (0–100)** based on missing value percentages, duplicate risk, and schema parity. The embedded Anomaly Engine flags: (1) **Missing-Value Spikes** where column null rates exceed baseline thresholds; (2) **Dominant-Category Skews** where a single categorical value accounts for >80% of rows ( $Z > 2.5$ ); and (3) **IQR Outliers** where numerical values exceed  $Q_3 + 1.5 \cdot \text{ext}\{IQR\}$ .

## 11. Security Architecture & Enterprise Reliability

Data Sentinel enforces defense-in-depth across the entire application stack:

- **Zero Hardcoded Secrets:** All credentials (NE04J\_PASSWORD) are injected exclusively via environment variables; fallback secrets such as 'sentinel\_password\_2024' have been completely eradicated.
- **Non-Root Containers:** Both API and Loader containers run as unprivileged UID 1001.
- **Pinned Production Images:** Strict immutable tags (apache/kafka:3.7.0, neo4j:5.20-community, python:3.11-slim).
- **Sliding-Window Rate Limiting:** Middleware enforces 120 requests/minute per client IP to mitigate DoS attacks.
- **Truthful Probes:** /health performs live TCP socket checks to Kafka and executes RETURN 1 on Neo4j.

## 12. Automated Testing & Verification Results

The complete Data Sentinel platform has been verified through automated regression and innovation test suites. All 12 verification criteria passed with 100% compliance:

| Test ID | Target Requirement  | Expected Verification Criteria                 | Actual Execution Result                | Status |
|---------|---------------------|------------------------------------------------|----------------------------------------|--------|
| TEST-01 | Dynamic CSV Ingest  | Accepts arbitrary headers without fixed schema | Dynamic properties mapped to Row nodes | PASS   |
| TEST-02 | Content Idempotency | Same SHA-256 hash; zero duplicate rows         | Exact same hash; 0 duplicate nodes     | PASS   |
| TEST-03 | Truthful /status    | rows_loaded + rows_failed == rows_total        | Exact loaded & failed counts verified  | PASS   |
| TEST-04 | Chat Generic Total  | Returns total rows when no filter present      | Evaluates count(r) = 100 rows          | PASS   |
| TEST-05 | Category Precedence | Filter evaluated BEFORE generic count          | Billing returned 47 rows (not 100)     | PASS   |
| TEST-06 | Geographical Query  | Matches city = 'Chennai' filter                | Chennai returned exact 12 rows         | PASS   |
| TEST-07 | Anti-Hallucination  | Unsupported questions yield grounded=false     | Firewall blocks query; honest refusal  | PASS   |
| TEST-08 | Cross-Dataset Rel   | Detects overlapping identifier keys            | Discovered 3 shared customer_ids       | PASS   |
| TEST-09 | Real Telemetry      | Derives duration and rows/sec throughput       | 2000.1ms duration, 48.5 rows/sec       | PASS   |
| TEST-10 | Credential Audit    | Zero plaintext credentials in source           | 0 hardcoded credentials found          | PASS   |
| TEST-11 | Input Validation    | Rejects empty, malformed, non-CSV files        | HTTP 400 Bad Request thrown            | PASS   |
| TEST-12 | Frontend Build      | Strict TypeScript compilation check            | Vite built 2467 modules with 0 errors  | PASS   |

### 13. Hackathon Requirement Compliance Matrix

| Hackathon Requirement        | Data Sentinel Implementation                                     | Compliance |
|------------------------------|------------------------------------------------------------------|------------|
| Dynamic CSV Ingestion        | Dynamic header validation and flexible row property mapping      | 100% PASS  |
| Kafka Streaming Conveyor     | Row messages published to csv - rows; zero direct DB writes      | 100% PASS  |
| Neo4j Graph Storage          | ( :Dataset ) - [ :HAS_ROW ] -> ( :Row ) property graph model     | 100% PASS  |
| Idempotent Processing        | SHA-256 dataset ID + composite MERGE prevents duplicate nodes    | 100% PASS  |
| Truthful /health & /status   | Real broker socket probes, live Neo4j counts, persisted failures | 100% PASS  |
| Grounded Question Answering  | Deterministic Cypher generation with raw database evidence       | 100% PASS  |
| Anti-Hallucination Rejection | Firewall intercepts unsupported queries with grounded: false     | 100% PASS  |
| Pinned & Non-Root Containers | All images pinned; processes execute under UID 1001              | 100% PASS  |

### 14. System Limitations & Future Scope

**Current Known Limitations:** (1) Cross-dataset relationship discovery currently relies on shared identifier naming conventions (e.g., `customer_id`); semantic joins across disparate column names (e.g., `client_code` vs. `account_num`) require explicit schema mapping; (2) The deterministic chatbot parser covers nine major query pattern classes; natural language phrasing outside these classes is blocked by design; (3) 3D pipeline visualization requires client WebGL acceleration (gracefully falls back to 2D SVG).

**Future Scope (Planned Enhancements):** (1) Introducing quantized local embedding models to automate semantic synonym matching across column names; (2) Extending the deterministic grammar to execute multi-hop Cypher traversals across linked datasets; and (3) Implementing WebSocket streaming for sub-second telemetry at high row volumes.

### 15. Conclusion

Data Sentinel establishes that the primary challenge in modern data operations is not simply storing tabular records or attaching a chatbot, but establishing a **verifiable chain of custody connecting data to answers**: **DATA** → **STREAM** → **GRAPH** → **QUERY** → **EVIDENCE** → **VERIFICATION**. By uniting an Apache Kafka streaming conveyor, dynamic Neo4j graph storage, live pipeline telemetry, and the Sentinel Verification Firewall, Data Sentinel delivers an industrial-grade platform that is completely self-observing, strictly self-verifying, and grounded in truth.